

Asumiendo que todo está en orden, aquí tienen una propuesta de organización para sus roles, etapas y entregables basada en el documento del curso:

Distribución de Roles (Para 5 integrantes)

Dado que la norma ISO/IEC 12207 requiere simular la relación "Cliente-Proveedor", un miembro del grupo debe asumir un rol externo al desarrollo técnico:

Integrante	Rol en la Simulación	Responsabilidades Clave
Integrante 1	El Cliente (Anglo Representative)	Define qué necesita el dashboard, evalúa las entregas, reporta "errores" simulados y pide la mejora final.
Integrante 2	Project Manager / Analista	Lidera la negociación con el cliente, redacta el contrato, el documento de requisitos y coordina el informe final.
Integrante 3	Desarrollador Frontend Principal	Estructura el código base de React en GitHub y documenta la arquitectura técnica (diagramas).
Integrante 4	Desarrollador de Soporte / Mantenimiento	Ayuda en el desarrollo y se encarga de implementar la "mejora" solicitada en la fase de mantenimiento.
Integrante 5	QA / Tester (Aseguramiento de Calidad)	Diseña el plan de pruebas, registra los bugs detectados y redacta el informe de validación.



Plan de Acción Paso a Paso

Para que no se abrumen, dividan el trabajo en las 5 etapas del estándar que solicita el proyecto:

Fase A: Inicio (Adquisición y Suministro)

- **Dinámica:** El **Cliente** le dice al equipo: *"Necesito un dashboard para visualizar los KPI de la operación Anglo"*.
- **Lo que deben hacer:** El **Analista** se reúne con el cliente y redacta los requisitos.

- **Entregables a preparar:**
 - Documento de especificación de requisitos (qué pantallas y gráficos tendrá el dashboard).
 - Un contrato simulado firmado por el Cliente y el Project Manager.

Fase B: Desarrollo (La base que ya tienen)

- **Dinámica:** Como ya tienen el repositorio **Dashboard_Anglo**, esta fase será más sencilla.
- **Lo que deben hacer:** El **Desarrollador Principal** sube el código a GitHub y se asegura de que compile. Crean la documentación de cómo está construido.
- **Entregables a preparar:**
 - Diagrama de arquitectura o diseño básico de componentes de React.
 - Código fuente estructurado en GitHub.
 - Un manual de usuario básico (ej. "Cómo iniciar el dashboard localmente", "Cómo leer las gráficas").

Fase C: Verificación y Validación (Pruebas)

- **Dinámica:** No podemos lanzar software sin probarlo.
- **Lo que debe hacer el QA:** Probar el dashboard de React en diferentes navegadores, verificar que los botones funcionen y que los datos se muestren bien.
- **Entregables a preparar:**
 - Un **Plan de Pruebas** sencillo (casos de prueba como: "Verificar que al hacer clic en el filtro de fechas la gráfica se actualice").
 - Un **Registro de errores** (pueden simular que encontraron 2 o 3 bugs en React y que luego los corrigieron).
 - Informe de validación.

Fase D: Operación Simulada (Feedback)

- **Dinámica:** Le entregan el dashboard al **Cliente** para que lo use.
- **Lo que deben hacer:** El cliente interactúa con la aplicación de React y da su opinión (ej. *"Me gusta el Dashboard, pero sería de gran ayuda tener un botón para exportar los reportes a PDF o un modo oscuro"*).

Fase E: Mantenimiento (La Mejora)

- **Dinámica:** Implementar el cambio solicitado por el cliente.
- **Lo que deben hacer:** El **Desarrollador de Mantenimiento** realiza el cambio solicitado en el código de React (ej. agregar el modo oscuro) y sube una nueva versión (v1.1) a GitHub.
- **Entregables a preparar:**
 - Registro de la solicitud de mejora.
 - Nueva versión en una rama de Git (**git branch feature-modos-oscuro**).
 - Informe técnico de mantenimiento.

El Informe Final y Reflexión

Al terminar, consolidarán todo en un único PDF final. Asegúrense de responder en equipo las tres preguntas clave de reflexión que plantea el profesor al final del documento (sobre la

facilidad/dificultad de la norma, el valor de la calidad y cómo adaptarlo a metodologías ágiles como Scrum).